

CARTES ANIMÉES

Document d'architecture

Choix techniques, architecture logicielle, modèle de données et règles métier

Projet	Cartes Animées – application web éducative pour enfants malentendants
Cadre	Projet de fin d'études – Mastère Expert en Systèmes d'Information (ESI2), Institut Limayrac
Auteur	Jean Lautraite
Tuteur technique	Stéphane Blusson
Tutrice pédagogique	Géraldine Boutan
Version	1.0
Date	29 août 2026

Table des matières

1. Présentation du projet.....	3
2. Du projet mobile à la webapp : justification du pivot.....	3
3. Architecture générale.....	4
4. Choix d'architecture : SPA découplée (API REST + Vue 3).....	5
4.1 Authentification : deux mécanismes assumés.....	5
5. Stack technique.....	5
6. Modèle de données.....	6
6.1 Identité et rôles.....	7
6.2 Enfants et suivi orthophonique.....	7
6.3 Contenu pédagogique.....	7
6.4 Progression et accès.....	7
6.5 Abonnement, facturation, rémunération.....	7
6.6 Paramétrage et historiques.....	7
7. Règles métier clés.....	8
7.1 Workflow de validation du contenu.....	8
7.2 Séries, progression et droits d'accès.....	9
7.3 Cycle de vie de l'abonnement.....	9
7.4 Rémunération des orthophonistes.....	10
7.5 Affiliation par code d'invitation.....	10
8. Intégrations.....	11
8.1 Stripe.....	11
8.2 E-mails et notifications.....	12
8.3 Stockage des médias.....	12
9. Sécurité.....	12
10. Choix et décisions de conception.....	13
11. Limites connues et perspectives.....	15
12. Annexe – Référence de l'API REST.....	16
Authentification.....	16
Enfants (rôle parent).....	16
Séries (rôle parent).....	16
Abonnements (rôle parent).....	16
Espace orthophoniste (rôle therapist).....	17
Notifications & profil (tous rôles connectés).....	17
Technique.....	17

1. Présentation du projet

Cartes Animées est une application web éducative destinée aux enfants malentendants : elle leur apprend à associer des mots à des cartes réunissant une animation dessinée, une animation réelle et un son, regroupées en séries thématiques. Trois profils interagissent : les **parents** (inscription, gestion des enfants, abonnement, usage avec l'enfant), les **orthophonistes** (suivi des patients, création de contenu, déblocage de séries) et l'**administrateur** (validation du contenu, gestion de la plateforme, rémunération des orthophonistes).

Le modèle économique est un abonnement mensuel **par enfant**, avec une commission reversée aux orthophonistes pour chaque patient actif qu'ils suivent sur l'application. Ce document présente l'architecture retenue, le modèle de données, les règles métier structurantes et la justification des choix de conception. Il reflète l'état réel du code au 29 août 2026.

2. Du projet mobile à la webapp : justification du pivot

Le besoin initial envisageait une application mobile native. Le choix s'est finalement porté sur une **application web responsive optimisée tablette**, pour des raisons assumées et cohérentes avec le contexte d'un PFE mené par un développeur unique :

- **Un seul développeur, un seul livrable** : une application mobile aurait exigé un développement iOS et Android (ou un framework cross-platform supplémentaire à apprendre), la gestion des stores et de leurs cycles de validation. La webapp offre un livrable unique, accessible immédiatement depuis n'importe quelle tablette, sans installation ni compte développeur Apple/Google.
- **Capitaliser sur la stack maîtrisée** : Laravel est utilisé quotidiennement en alternance. Construire le back-end, le back-office et l'API sur Laravel 12 garantissait productivité et qualité, plutôt que de répartir l'effort d'apprentissage sur une pile mobile entière.
- **Faire du projet un levier de progression ciblé** : le développement front-end était le point à renforcer. Le pivot concentre l'apprentissage sur Vue.js et l'écosystème front moderne (Vite, Pinia, Tailwind), une compétence directement réutilisable professionnellement.
- **L'usage cible ne l'exigeait pas** : l'application s'utilise à la maison ou en cabinet, sur tablette avec connexion : ni capteurs natifs, ni notifications push, ni mode hors-ligne n'étaient requis au périmètre du PFE.

Le pivot ne ferme aucune porte : l'API REST découplée (section 4) permettrait demain de développer une application mobile native consommant exactement les mêmes services, sans toucher au back-end.

3. Architecture générale

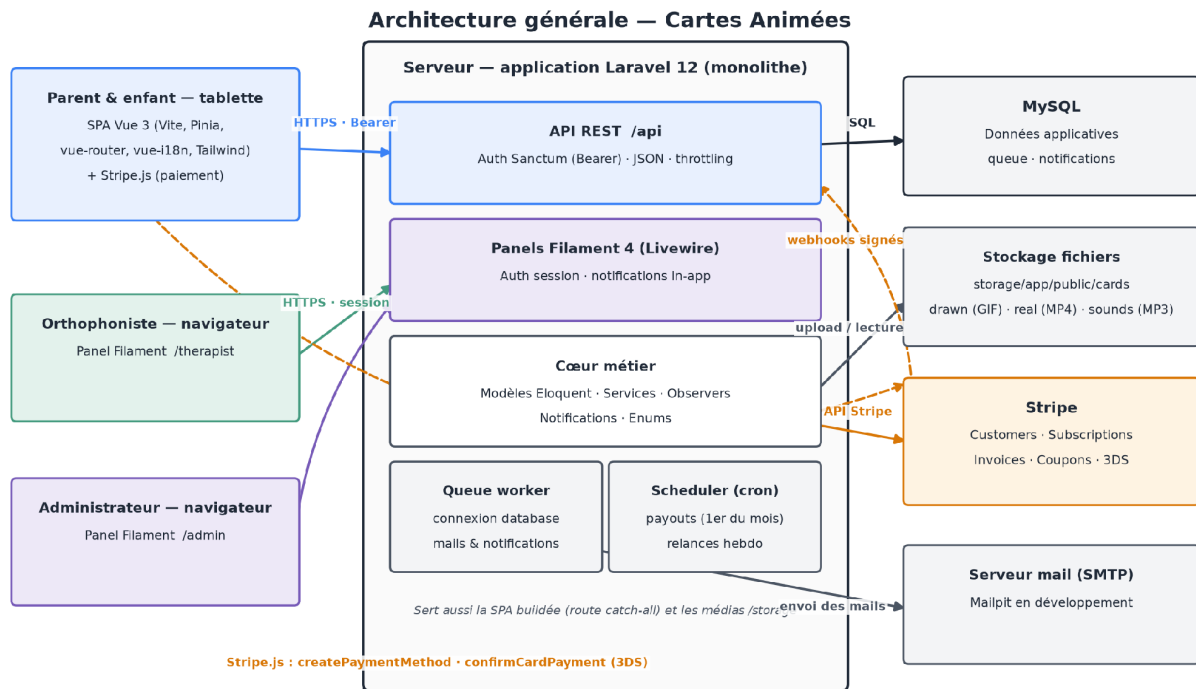


Figure 1 — Architecture générale.

L'application est un **monolithe Laravel 12** qui expose trois surfaces distinctes :

- **L'API REST (/api)** : consommée par la SPA Vue 3. Authentification par jeton Bearer (Laravel Sanctum), réponses JSON normalisées, limitation de débit (throttling).
- **Les panels Filament 4 (/admin, /therapist)** : back-offices complets construits avec Filament (Livewire), authentification par session, notifications in-app en base de données. Chaque panel a son thème et ses ressources propres.
- **La SPA Vue 3** : compilée par Vite, servie par Laravel via une route « attrape-tout » qui renvoie la page hôte de l'application. Le routage applicatif est ensuite assuré côté client par Vue Router.

Autour du cœur métier (modèles Eloquent, services, observers, notifications), deux processus d'appoint tournent en continu : le **worker de queue** (connexion database – envoi des e-mails et notifications) et le **planificateur** (calcul mensuel des virements orthophonistes le 1^{er} du mois, relance hebdomadaire des parents inactifs). Les dépendances externes sont MySQL, l'API Stripe (avec retour par webhooks signés) et un serveur SMTP.

i Un seul déploiement sert les trois surfaces : pas de serveur front séparé, pas de CORS, une seule chaîne de build. Les médias pédagogiques (GIF, MP4, MP3) sont stockés sur le disque public du serveur et servis statiquement sous /storage/cards.

4. Choix d'architecture : SPA découplée (API REST + Vue 3)

Trois approches étaient possibles pour le front parent/enfant au-dessus de Laravel :

Approche	Principe	Pourquoi pas / pourquoi
Blade + Livewire	Rendu serveur, interactivité via Livewire (l'approche des panels Filament)	Adaptée aux back-offices, mais peu adaptée à une interface tactile riche type player d'animations ; n'aurait pas fait progresser la compétence front visée
Inertia.js	Vue rendu par Laravel sans API formelle	Couplage fort front/back : pas de contrat d'API réutilisable pour un futur client mobile
SPA + API REST (retenue)	Vue 3 autonome consommant une API Sanctum	Séparation nette des responsabilités, contrat d'API documenté et testable (Postman), réutilisable tel quel par une future app mobile ; correspond à l'objectif pédagogique du projet

Le découplage retenu est **logique, pas physique** : la SPA est buildée par Vite et servie par la même application Laravel (route attrape-tout). On garde ainsi les bénéfices du découplage (API propre, front autonome) sans les coûts d'un second déploiement (CORS, domaine dédié, double pipeline).

4.1 Authentification : deux mécanismes assumés

- **SPA → API : jetons Sanctum (Bearer).** Le jeton est délivré à la connexion, stocké côté client et envoyé dans l'en-tête Authorization. Une expiration d'inactivité de 2 heures est appliquée côté client.
- **Panels Filament : sessions Laravel classiques,** adaptées à un back-office rendu serveur (CSRF, cookies).

Le rôle de l'utilisateur (parent, therapist, admin – exclusif) gouverne l'accès : la SPA est réservée au parcours parent, chaque panel vérifie le rôle à l'entrée (l'admin a également accès au panel orthophoniste), et chaque endpoint de l'API contrôle la possession des ressources (un parent n'accède qu'à ses enfants, un orthophoniste qu'à ses patients).

5. Stack technique

Couche	Technologie	Rôle
Back-end	Laravel 12 (PHP ≥ 8.2)	API REST, logique métier, planification, queue
Back-offices	Filament 4	Panels admin et orthophoniste (ressources, widgets, notifications in-app)
Auth API	Laravel Sanctum 4	Jetons personnels (Bearer)

Couche	Technologie	Rôle
Paieement	stripe/stripe-php 20 + Stripe.js (@stripe/stripe-js 9)	Abonnements, 3D Secure, webhooks, coupons
Front	Vue 3.5 (Composition API, script setup)	SPA parent/enfant
Build front	Vite 7 + laravel-vite-plugin	Compilation SPA et thèmes Filament
État & routage	Pinia 3 · Vue Router 5	Stores (auth, enfants, UI) · navigation client
i18n	vue-i18n 11 (front) · lang/ (back)	Français livré, anglais préparé
Styles	Tailwind CSS 4	Interface tablette, thèmes parent/enfant
Base de données	MySQL 8	Données, queue, notifications
Impersonation	stechstudio/filament-impersonate	Support : l'admin se connecte en tant qu'un utilisateur
E-mails (dev)	Mailpit	Boîte de réception locale

6. Modèle de données

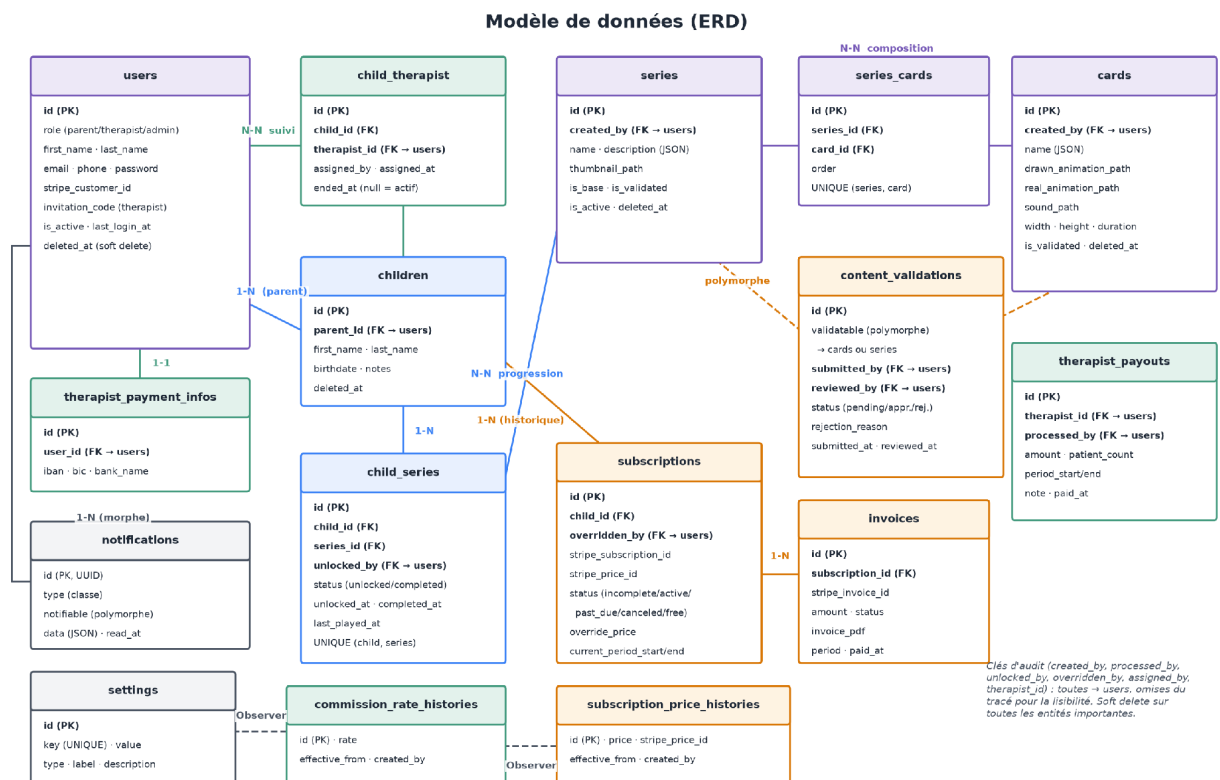


Figure 2 – Modèle de données (Les champs d'audit vers users sont notés dans Les entités).

Le modèle s'organise en six domaines :

6.1 Identité et rôles

`users` est la table d'authentification unique ; elle porte le rôle (parent / therapist / admin, exclusif), l'état d'activation (`is_active`, blocage par l'admin), la date de dernière connexion (`last_login_at`, base des relances d'inactivité) et deux champs spécialisés : `stripe_customer_id` (parents) et `invitation_code` (orthophonistes, unique et régénérable). Les coordonnées bancaires des orthophonistes sont isolées dans `therapist_payment_infos` (relation 1-1) : une table à part, à l'accès restreint au panel admin et à son propriétaire.

6.2 Enfants et suivi orthophonique

`children` rattache chaque enfant à un unique compte parent (1-N). La relation orthophoniste-enfant est N-N via `child_therapist` : un enfant peut être suivi par plusieurs orthophonistes (suivi partagé), un orthophoniste suit plusieurs enfants. La fin de suivi ne supprime rien : `ended_at` est horodaté et la ligne conservée, ce qui préserve l'historique et permet un calcul de rémunération exact sur les périodes passées.

6.3 Contenu pédagogique

`cards` est l'unité de contenu : un nom multilingue (JSON), trois médias obligatoires (animation dessinée, animation réelle, son) et des métadonnées partagées (dimensions, durée). `series` regroupe des cartes via la table pivot `series_cards` qui porte l'ordre de lecture ; une carte peut appartenir à plusieurs séries. Trois drapeaux gouvernent la visibilité d'une série : `is_validated` (issue du workflow de validation), `is_active` (interrupteur de l'admin) et `is_base` (série d'essai accessible sans abonnement).

6.4 Progression et accès

`child_series` matérialise l'accès d'un enfant à une série : **pas de ligne = pas d'accès**. La ligne porte qui a débloqué (`unlocked_by`, null pour les séries de base débloquées automatiquement), le statut (`unlocked` / `completed`), les horodatages et `last_played_at`, mis à jour à chaque ouverture de la série par l'enfant – ce qui alimente les tableaux de bord des orthophonistes.

6.5 Abonnement, facturation, rémunération

`subscriptions` est liée à l'enfant (1-N : un seul abonnement en cours, historique conservé). Elle porte les identifiants Stripe (null pour un abonnement gratuit), le statut (`incomplete` / `active` / `past_due` / `canceled` / `free`), l'`override_price` (remise ou gratuité décidée par l'admin) et la période courante. `invoices` trace chaque cycle de facturation (miroir local des factures Stripe, PDF inclus). `therapist_payouts` enregistre les virements mensuels calculés pour les orthophonistes, avec le nombre de patients rémunérateurs et la période – un instantané qui fige la base de calcul.

6.6 Paramétrage et historiques

`settings` stocke les paramètres administrables en clé/valeur typée (prix de l'abonnement, taux de commission), modifiables sans déploiement. Deux tables d'historique en découlent automatiquement, alimentées par un observer à chaque modification : `commission_rate_histories` et `subscription_price_histories` (cette dernière

portant aussi le nouvel identifiant de prix créé chez Stripe). Enfin, notifications est la table native Laravel, utilisée pour les notifications in-app des panels.

7. Règles métier clés

7.1 Workflow de validation du contenu

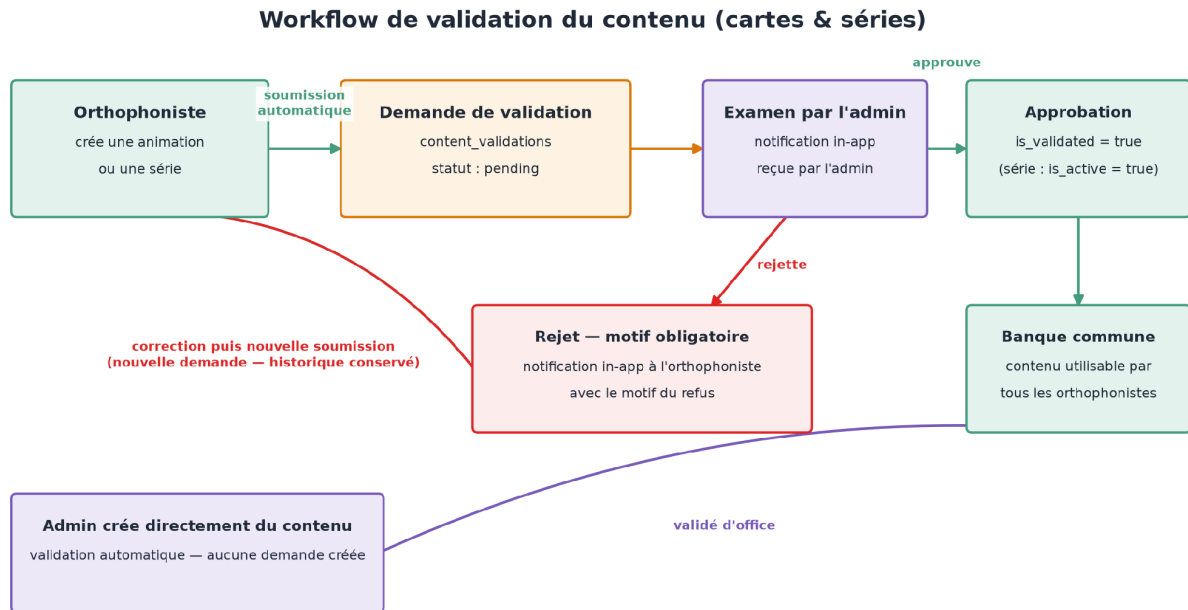


Figure 3 — Workflow de validation.

- Toute carte ou série créée par un orthophoniste est soumise automatiquement à validation : une ligne « pending » est créée dans content_validations et l'admin est notifié in-app.
- L'admin approuve (le contenu devient utilisable ; une série est aussi activée) ou rejette – le motif est alors obligatoire et notifié à l'orthophoniste.
- Un contenu rejeté peut être corrigé et resoumis : chaque soumission crée une nouvelle ligne, sans contrainte d'unicité – l'historique complet des tentatives est conservé.
- Le contenu créé par l'admin est validé d'office, sans passer par le circuit (l'admin serait son propre validateur).
- La validation est portée par une relation polymorphique : une seule table couvre les cartes et les séries (voir décision n° 6).
- Une fois validé, le contenu rejoint la banque commune : toutes les cartes servent à tous les orthophonistes pour composer des séries, et toute série validée peut être attribuée à n'importe quel patient.

7.2 Séries, progression et droits d'accès

- À la création d'un profil enfant, un **observer** débloquent automatiquement toutes les séries de base validées et actives (`unlocked_by = null`).
- Seul un orthophoniste référent de l'enfant (ou l'admin) peut débloquent une série supplémentaire, la marquer complétée ou la retirer ; les séries de base ne sont pas retirables.
- Règle d'accès appliquée par l'API : avec un abonnement `active` ou `free`, l'enfant accède à toutes ses séries débloquées ; sans abonnement en règle (aucun, `incomplete`, `past_due`, `canceled`), l'API ne renvoie que les séries de base. La régularisation d'un paiement restaure l'accès automatiquement, sans action manuelle.
- L'enfant conserve ses séries débloquées même si le suivi orthophonique s'arrête ; seul le déblocage de nouvelles séries requiert un orthophoniste actif.

7.3 Cycle de vie de l'abonnement

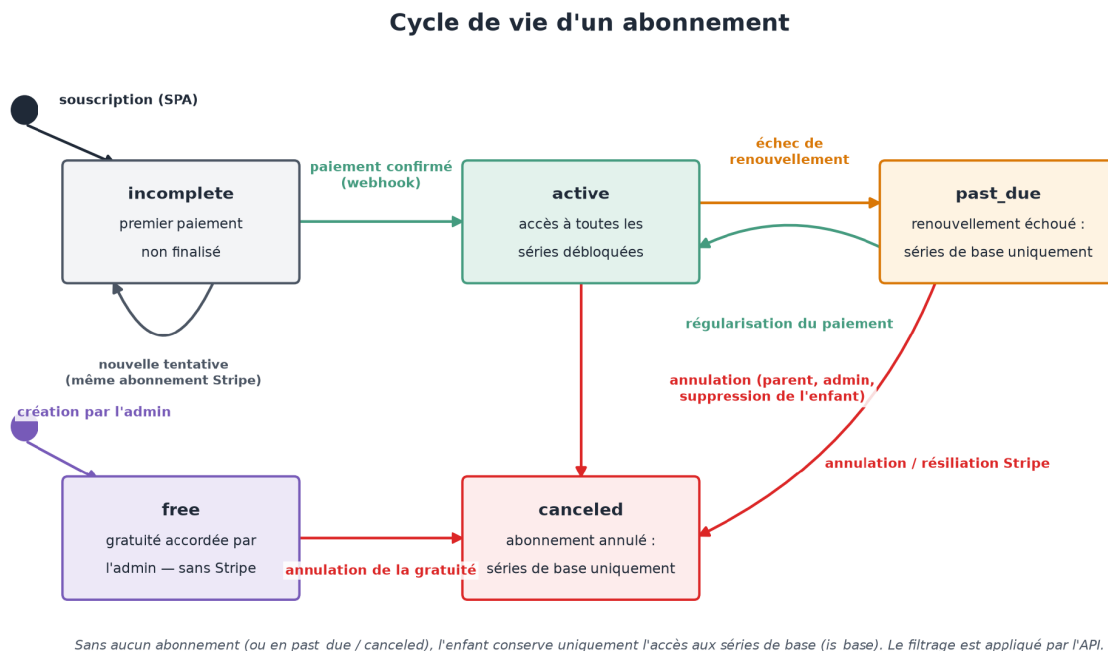


Figure 4 – États d'un abonnement.

L'abonnement est souscrit **par enfant** – un parent de deux enfants gère deux abonnements indépendants. Le prix courant est le dernier de `subscription_price_histories` ; l'admin peut appliquer une remise (coupon Stripe) ou la gratuité totale (statut `free`, sans objet Stripe). Points notables du cycle :

- `incomplete` : le premier paiement n'a pas abouti (3DS abandonnée, carte refusée). L'abonnement Stripe existe déjà : une nouvelle tentative de paiement réutilise le même abonnement au lieu d'en créer un second.
- Un échec sur la **première** facture ne passe pas l'abonnement en `past_due` (ce statut suppose une période active antérieure) et ne déclenche pas l'e-mail d'échec de renouvellement – distinction gérée dans le traitement du webhook.

- `past_due` : un renouvellement a échoué – e-mail au parent, accès restreint aux séries de base, bouton de régularisation dans l'application.
- La suppression du profil d'un enfant annule proprement son abonnement Stripe (observer sur la suppression).

7.4 Rémunération des orthophonistes

- Le 1^{er} de chaque mois, la commande planifiée `payouts:calculate` calcule pour chaque orthophoniste actif : nombre de patients rémunérateurs × taux de commission.
- Un patient est **rémunérateur** si son suivi était actif sur la période et si son abonnement est actif et payant (les gratuités totales sont exclues ; une remise partielle reste rémunératrice). En suivi partagé, chaque orthophoniste perçoit la commission complète.
- Le taux appliqué est celui en vigueur à la fin de la période, résolu via `commission_rate_histories` – un choix de simplification assumé : l'historique complet des taux permet en contrepartie d'auditer précisément tout calcul passé (voir décision n° 15).
- Le virement est généré « en attente », notifié à l'orthophoniste, puis exécuté manuellement par l'admin qui le marque payé. Le calcul est idempotent : une période déjà calculée n'est jamais dupliquée, et un mode simulation (`--dry-run`) permet de vérifier sans écrire.

7.5 Affiliation par code d'invitation

- Chaque orthophoniste possède un code unique (8 caractères), généré à la création de son compte, communiqué dans son e-mail de bienvenue, et régénérable à volonté (l'ancien code devient immédiatement inopérant).
- Le parent initie toujours la liaison : il saisit le code sur la fiche de son enfant. La liaison est immédiate, sans validation de l'orthophoniste ; celui-ci est notifié in-app et le parent par e-mail.
- Le mécanisme couvre tous les cas du métier : parent inscrit sans orthophoniste, enfants d'une même fratrie suivis par des praticiens différents, suivi partagé à plusieurs sur un même enfant.

8. Intégrations

8.1 Stripe

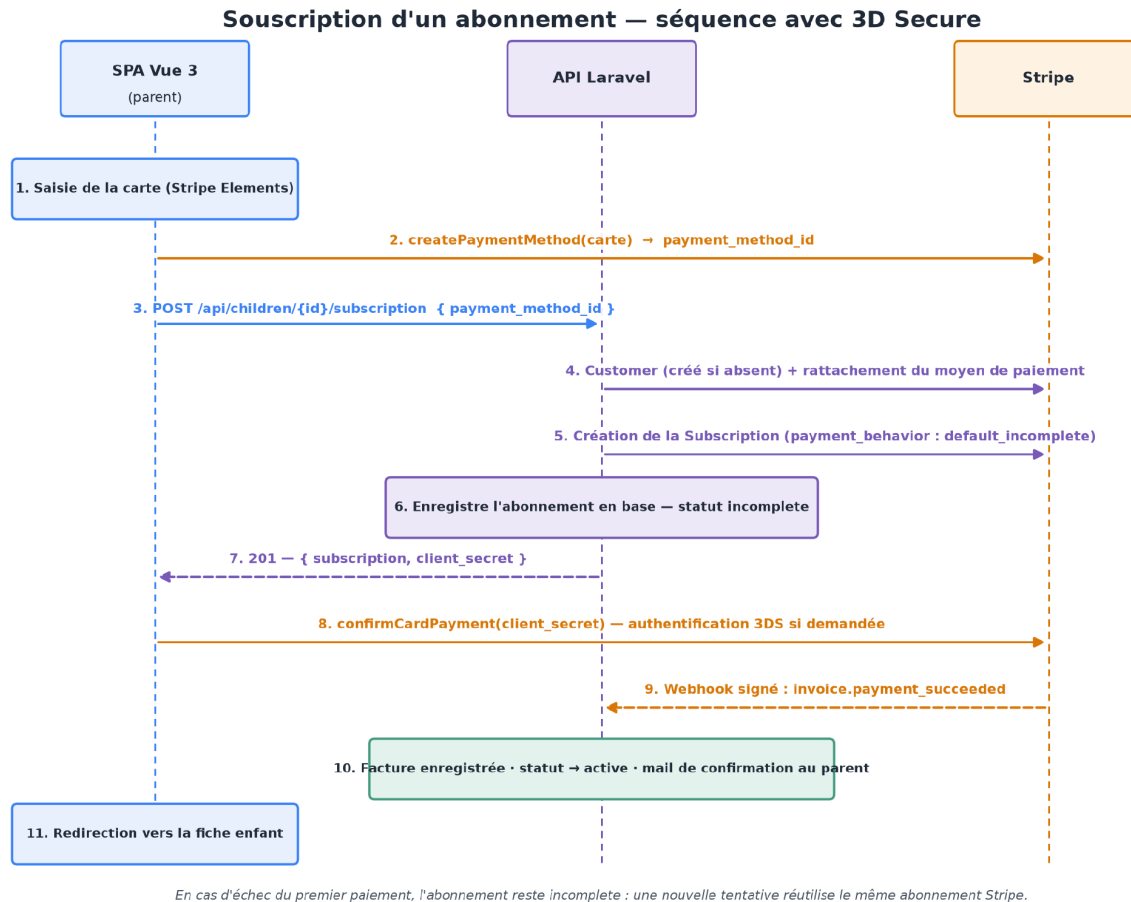


Figure 5 — Séquence de souscription avec 3D Secure.

La souscription suit le schéma « Payment Intents » de Stripe : la carte est tokenisée côté client par Stripe.js (les numéros ne transitent **jamais** par le serveur), l'API crée l'abonnement en mode `default_incomplete` et renvoie le `client_secret` du paiement en attente ; la SPA confirme alors le paiement, ce qui déclenche l'authentification 3D Secure si la banque l'exige. La confirmation d'encaissement arrive ensuite de façon asynchrone par webhook signé, qui active l'abonnement et enregistre la facture.

- **Adaptation à l'API Stripe 2026** : la version courante de l'API ne relie plus directement la facture à son `PaymentIntent` ; le service Stripe du projet retrouve donc le paiement en attente en listant les `PaymentIntents` du client et en sélectionnant celui en statut confirmable – encapsulé dans `StripeService`, invisible du reste du code.
- **Prix historisés** : changer le prix dans les paramètres crée un nouveau `Price` Stripe et une entrée d'historique. Les abonnements existants conservent leur prix d'origine (`grandfathering`) ; les nouvelles souscriptions utilisent le dernier prix.
- **Remises** : matérialisées par des coupons Stripe appliqués à l'abonnement (montant fixe, durée illimitée), retirables à tout moment ; la gratuité totale, elle, ne crée aucun objet Stripe.

- **Erreurs de carte** : les codes de refus Stripe (fonds insuffisants, carte expirée, CVC incorrect...) sont traduits en messages utilisateur explicites par l'API.

8.2 E-mails et notifications

Deux canaux assumés selon la surface : les utilisateurs de la SPA (parents) sont joints par **e-mail** (bienvenue, paiements, suivi, relance d'inactivité après deux semaines sans connexion) ; les utilisateurs des panels (orthophonistes, admin) reçoivent des **notifications in-app** via le système de notifications base-de-données de Filament (cloche, interrogation périodique). Tous les envois transitent par la queue pour ne jamais bloquer une requête HTTP.

8.3 Stockage des médias

Les médias pédagogiques sont stockés sur un disque Laravel dédié (« cards ») pointant vers `storage/app/public/cards`, organisé en trois sous-dossiers : `drawn/` (GIF), `real/` (MP4) et `sounds/` (MP3). À la création d'une carte, les fichiers téléversés sont renommés automatiquement selon le motif `{slug}_{type}_{timestamp}.{ext}` – noms stables, lisibles et sans collision. Les fichiers sont servis statiquement (lien symbolique `/storage`), sans passer par PHP : le player charge GIF, vidéos et sons comme de simples ressources HTTP.

9. Sécurité

- **Authentification et rôles** : jetons Sanctum pour l'API, sessions pour les panels ; rôle unique et exclusif par compte, vérifié à l'entrée de chaque panel ; un compte désactivé (`is_active = false`) ne peut plus se connecter.
- **Contrôle de possession systématique** : chaque endpoint vérifie que la ressource appartient bien à l'appelant (un parent n'atteint que ses enfants, un orthophoniste que ses patients au suivi actif) – les identifiants devinés renvoient 403/404.
- **Limitation de débit** : 5 requêtes/minute sur l'inscription et la connexion, 3/minute sur la réinitialisation de mot de passe, 60/minute sur l'API authentifiée.
- **Mots de passe et récupération** : hachage bcrypt, minimum 8 caractères ; la demande de réinitialisation répond toujours positivement, qu'un compte existe ou non (pas d'énumération d'e-mails) ; lien à jeton expirable envoyé par e-mail.
- **Paiements** : aucune donnée de carte côté serveur (`tokenisationStripe.js`) ; webhooks vérifiés par signature (`STRIPE_WEBHOOK_SECRET`) ; journal Stripe dédié.
- **Données** : soft delete généralisé (traçabilité, pas de cascade destructive) ; coordonnées bancaires isolées dans une table dédiée ; validation systématique des entrées (Form Requests / règles de validation).
- **Impersonation contrôlée** : réservée à l'admin, visible (bandeau permanent côté parent, lien de retour côté panel), avec un jeton dédié de portée session.

10. Choix et décisions de conception

Registre des décisions structurantes, avec leur justification – dans l'état actuel du code.

N°	Décision	Justification
1	Séparer parent et enfant (table children)	Un compte parent gère plusieurs enfants ; l'abonnement, le suivi et la progression sont portés par l'enfant, pas par le compte. Plus juste pour le métier (fratries) et plus naturel côté UX.
2	Clés étrangères nommées par rôle métier	parent_id, therapist_id, created_by, unlocked_by... plutôt que user_id partout : le schéma se lit seul, et les relations Eloquent (parent(), creator()...) restent expressives.
3	Coordonnées bancaires isolées (therapist_payment_infos)	IBAN/BIC hors de la table users : accès restreint, exposition minimale dans les listes et exports. (Le chiffrement au repos reste une limite connue, voir section 11.)
4	Métadonnées partagées GIF/vidéo	Les deux animations d'une carte respectent les mêmes normes : un seul jeu width/height/duration suffit, saisi une fois.
5	Multilingue par champs JSON	name et description stockent {"fr": ..., "en": ...} : extensible sans migration, natif chez Laravel (cast array), administrable en back-office. Le front est câblé sur vue-i18n : seul le français est livré, l'anglais est préparé.
6	Validation polymorphe (content_validations)	Cartes et séries suivent le même workflow : une seule table polymorphe au lieu de deux tables jumelles – moins de duplication, un seul écran d'administration.
7	Séries de base débloquent par observer	À la création d'un enfant, l'observer insère les lignes child_series des séries de base : l'essai gratuit est immédiat, sans action manuelle ni code dupliqué dans les contrôleurs.
8	Contenu de l'admin auto-validé	L'admin serait son propre validateur : is_validated = true directement, aucune demande créée – le circuit de validation reste réservé aux soumissions des orthophonistes.
9	Historique complet des soumissions	Pas de contrainte d'unicité sur content_validations : chaque resoumission est une nouvelle ligne, l'historique des refus et corrections est conservé.
10	override_price plutôt qu'une table de remises	Un champ unique couvre remise (valeur) et gratuité (0) ; null = prix normal. Complété en pratique par les coupons Stripe pour que la remise s'applique réellement à la facturation (décision n° 19).
11	Virements gérés en back-office, sans Stripe Connect	therapist_payouts trace montants, effectifs et périodes ; l'admin vire manuellement et marque payé. Stripe Connect aurait été surdimensionné

N°	Décision	Justification
		pour le volume visé – la traçabilité est assurée, l'automatisation reste possible plus tard.
12	Soft delete généralisé	Aucune donnée réellement supprimée : suppression d'un orthophoniste sans impact sur ses patients, historique de suivi conservé, auditabilité complète.
13	Notifications sur le système natif Laravel	Table notifications standard, canaux mail et database selon la surface – pas de mécanisme maison ; un canal supplémentaire (push...) resterait branchable sans toucher à la structure.
14	settings en clé/valeur typée	Prix et taux modifiables par l'admin sans déploiement ; le champ type garantit le cast correct, label et description rendent l'écran de paramétrage autoportant.
15	Historisation du taux de commission	Chaque changement de taux crée une ligne (observer sur settings) : tout calcul de paie passé est auditable. Le calcul mensuel applique le taux en vigueur en fin de période – simplification assumée, l'historique permettant de justifier ou recalculer finement si besoin.
16	Affiliation par code d'invitation	Le parent initie la liaison en saisissant le code du praticien : couvre l'inscription sans référent, les fratries multi-praticiens et le suivi partagé, sans annuaire public d'orthophonistes ni validation croisée.
17	SPA servie par Laravel (découplage logique)	Le front Vue est découplé par l'API mais hébergé par la même application : un seul déploiement, pas de CORS, tout en gardant un contrat d'API réutilisable par un futur client mobile.
18	Historisation des prix d'abonnement (grandfathering)	Changer le prix crée un nouveau Price Stripe et une entrée d'historique : les abonnés existants conservent leur tarif, les nouveaux paient le prix courant – équitable et sans migration des abonnements en cours.
19	Remises via coupons Stripe	La remise décidée en back-office est matérialisée par un coupon appliqué à l'abonnement Stripe : c'est la facturation réelle qui est réduite, pas seulement une valeur d'affichage.
20	Impersonation pour le support	L'admin peut ouvrir l'application au nom d'un parent (pont de jeton dédié, bandeau visible, retour en un clic) ou le panel d'un orthophoniste : diagnostic des problèmes utilisateurs sans demander leurs identifiants.

11. Limites connues et perspectives

Points connus, documentés et volontairement hors du périmètre livré :

Point	Statut / perspective
Signature du webhook Stripe	Fonctionnelle en sandbox ; créer l'endpoint et le secret définitifs lors d'un passage en production réelle
Mode hors-ligne pour les séries de base	Non implémenté (nice-to-have) ; nécessiterait une PWA avec cache des médias
Mention RGPD (rétention de l'historique)	À ajouter aux documents légaux (CGU)
Route de test /api/test/payment-method	Laissée volontairement pour la démonstration ; à retirer en production réelle
Internationalisation	Architecture prête (JSON en base, vue-i18n, fichiers lang) ; seul le français est livré
Application mobile native	Perspective : l'API REST est réutilisable telle quelle par un client iOS/Android

12. Annexe – Référence de l'API REST

Toutes les routes sont préfixées par `/api`. Sauf mention « public », elles exigent l'en-tête `Authorization: Bearer {token}`. Les routes authentifiées sont limitées à 60 requêtes/minute.

Authentification

Méthode	Route	Accès	Description
POST	<code>/auth/register</code>	public	Inscription d'un parent (rôle parent assigné) – 5 req/min
POST	<code>/auth/login</code>	public	Connexion (tous rôles) – 403 si compte désactivé – 5 req/min
POST	<code>/auth/logout</code>	connecté	Révoque le jeton courant
GET	<code>/auth/me</code>	connecté	Utilisateur courant
POST	<code>/auth/password/forgot</code>	public	Demande de réinitialisation (répond toujours 200) – 3 req/min
POST	<code>/auth/password/reset</code>	public	Réinitialisation avec jeton – 3 req/min

Enfants (rôle parent)

Méthode	Route	Description
GET	<code>/children</code>	Enfants du parent connecté
POST	<code>/children</code>	Créer un enfant (déclenche le déblocage des séries de base)
GET / PUT / DELETE	<code>/children/{id}</code>	Consulter, modifier, supprimer (soft delete + annulation de l'abonnement)
POST	<code>/children/{id}/therapist</code>	Affilier un orthophoniste via code d'invitation
DELETE	<code>/children/{id}/therapist/{therapistId}</code>	Mettre fin à un suivi

Séries (rôle parent)

Méthode	Route	Description
GET	<code>/children/{id}/series</code>	Séries débloquées de l'enfant (filtrées selon l'abonnement)
GET	<code>/children/{id}/series/{seriesId}</code>	Détail d'une série + cartes ordonnées avec URLs des médias (met à jour <code>last_played_at</code>)

Abonnements (rôle parent)

Méthode	Route	Description
GET	/children/{id}/subscription	Dernier abonnement de l'enfant
POST	/children/{id}/subscription	Souscrire (payment_method_id) – renvoie un client_secret si confirmation/3DS requise ; réutilise l'abonnement existant en incomplete/past_due
DELETE	/children/{id}/subscription	Annuler l'abonnement
GET	/subscription/price	Prix mensuel en vigueur

Espace orthophoniste (rôle therapist)

Méthode	Route	Description
GET	/therapist/patients	Patients au suivi actif
GET	/therapist/patients/{id}	Détail d'un patient
POST	/therapist/invitation-code	Régénérer le code d'invitation
POST	/therapist/patients/{id}/series/{seriesId}	Débloquer une série pour un patient
PATCH	/therapist/patients/{id}/series/{seriesId}	Marquer une série complétée
DELETE	/therapist/patients/{id}/series/{seriesId}	Retirer une série

Notifications & profil (tous rôles connectés)

Méthode	Route	Description
GET	/notifications	Notifications de l'utilisateur
PATCH	/notifications/{id}	Marquer lue
PATCH	/notifications/read-all	Tout marquer lu
GET / PUT	/profile	Consulter / modifier le profil (prénom, nom, téléphone)
PATCH	/profile/password	Changer le mot de passe (mot de passe actuel requis)

Technique

Méthode	Route	Description
POST	/stripe/webhook	Réception des événements Stripe (signature vérifiée) – usage interne
GET	/test/payment-method	Génère un moyen de paiement de test (sandbox) – route de démonstration, à retirer en production

i La gestion des informations bancaires des orthophonistes ne passe plus par l'API : elle s'effectue dans le panel orthophoniste (Paie & administratif → Informations bancaires).